

Low-Overhead Dirty Access Tracking in GPU-UVM

Aditi Khandelia Arush Upadhyaya Kushagra Srivastava Sankalp Mittal Vidhi Jain

CS614 : Linux Kernel Programming
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

Overview

- 1 Introduction
- 2 Motivation
- 3 Implementation
- 4 Design Choices
- 5 Data Structures and Associated Operations
- 6 Testing and Analysis
- 7 Future Work
- 8 Conclusion

- NVIDIA's **Unified Virtual Memory (UVM)** subsystem underpins cudaMallocManaged and manages demand-paging between host and device via a hardware fault-driven path
- UVM exposes no interface for querying which pages a process has modified; any system requiring this must conservatively copy the **entire** GPU memory footprint
- We implement **per-process dirty page tracking** directly within the open-source NVIDIA UVM kernel module, augmenting the write-fault servicing path without additional hardware overhead
- Each recorded write carries a **nanosecond-resolution timestamp**, enabling precise ordering of writes across epochs and across CPU-GPU phases
- A procfs-based interface exposes start, stop, pause, resume, and atomic cutover snapshot operations to a privileged user with **page-level granularity**

Motivation

Why Dirty Tracking?

- The Linux kernel tracks CPU dirty pages via soft-dirty bits in page-table entries, readable through `/proc/PID/pagemap`; **no equivalent exists for GPU-managed memory**
- Live process migration relies on identifying exactly which pages were written each round; without GPU dirty tracking, the entire GPU memory footprint must be copied conservatively on every iteration
- Fault-tolerant ML training requires knowing which model-parameter pages were updated to enable **incremental checkpointing** and avoid full restart on failure
- In heterogeneous CPU-GPU workloads with alternating compute phases, only GPU-written pages need to be synchronised back to the host; tracking eliminates unnecessary transfers
- UVM already intercepts every GPU page fault for demand-paging, making the fault path a **natural and zero-hardware-cost** instrumentation point

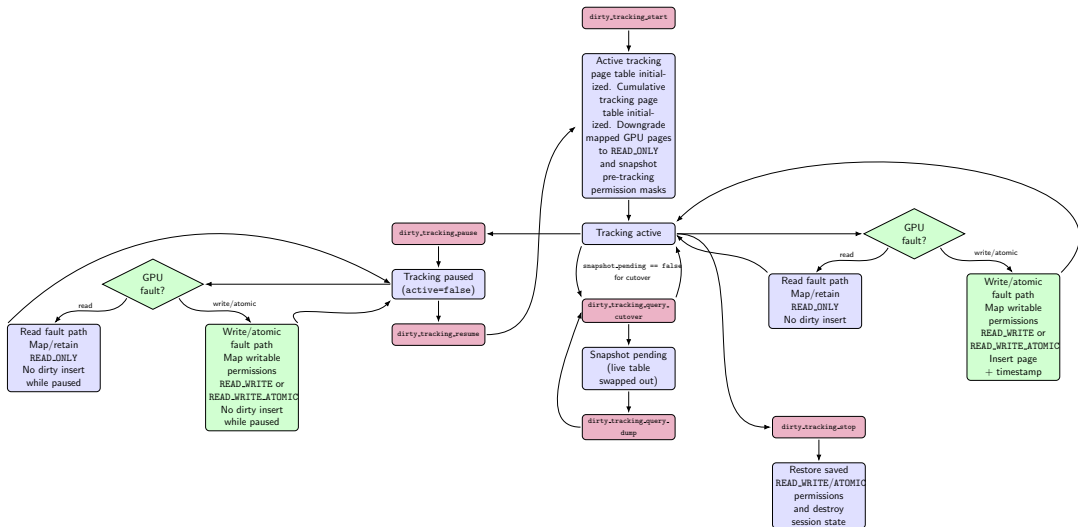
Motivation

Limitations of Existing Approaches

- **Full memory copy:** The only correct option today without tracking; conservatively transfers the entire allocation regardless of how little was actually written
- **Application-level bookkeeping:** Requires instrumenting every kernel call site to record writes; not transparent, breaks with third-party or compiled-only code
- **Sampling-based profilers (CUPTI, nvprof):** Impose high overhead, capture aggregate statistics rather than per-page write identity, and require profiling mode which is unavailable in production
- **Software checksumming:** Hashing memory regions to detect changes requires reading the entire allocation on every epoch boundary, producing overhead proportional to allocation size rather than write density
- **No epoch or timestamp support:** None of the above provide a mechanism to atomically snapshot the dirty set at an epoch boundary or order writes in time, both of which are required for correct iterative migration

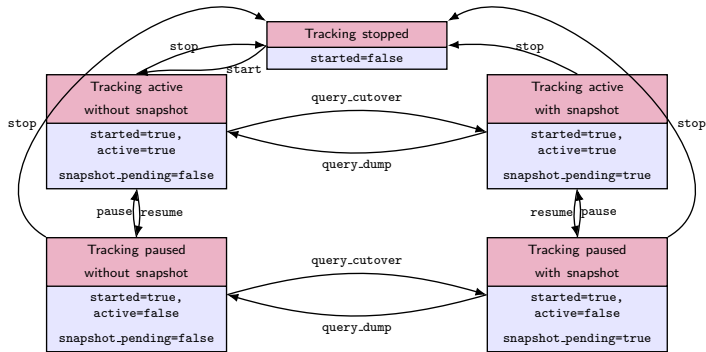
Implementation

End-to-End Workflow



Implementation

State Transition Diagram



All lifecycle operations are serialised by `uvm_dirty_lifecycle_lock`. Transitions that violate owner checks, state preconditions, or snapshot guards are rejected by the procs handlers.

Implementation

Permission Lifecycle (1/2)

Phase	GPU Page Permission	Dirty Recording
start / resume	Save <code>pte_bits[WRITE/ATOMIC]</code> into <code>dirty_downgraded_pages</code> ; revoke write perms → <code>READ_ONLY</code>	Tables initialised; <code>active=true</code>
Read fault	Forced <code>READ_ONLY</code> (<code>uvm_va_block.c</code>)	Not inserted
Write fault	Restored to <code>READ_WRITE / ATOMIC</code> (<code>uvm_gpu_replayable_faults.c</code>)	<code>page_num + ktime_get_ns()</code> inserted (first-write-wins)
Prefetch	GPU-migrated pages mapped <code>READ_ONLY</code> (<code>uvm_migrate.c</code>)	Not inserted; write fault records on first write
pause	Write faults still grant <code>READ_WRITE/ATOMIC</code> for forward progress	Not inserted (<code>active=false</code>); query barrier drains fault buffer first

Implementation

Permission Lifecycle (2/2)

Phase	GPU Page Permission	Dirty Recording
resume	Re-runs downgrade sequence; READ_ONLY re-armed without reinitialising tables	Recording resumes; active=true
cutover	No change	Live → snapshot under query barrier; fresh live table installed; snapshot_pending=true
dump	No change	Delta: emit snapshot. Cumulative: merge into cumulative (first-write-wins), emit union
stop	Walk VA blocks; restore dirty_downgraded_pages masks; exact pre-tracking perms recovered	All tables destroyed; state reset

Implementation

Procs Interface: Lifecycle Controls

dirty_tracking_start

```
echo "<PID> delta|cumulative" > /proc/driver/nvidia-vm/dirty_tracking_start
```

Initialises live table; downgrades all GPU PTEs to READ_ONLY and saves per-page PTE masks; sets active=true. Mode selects delta (per-epoch diff) or cumulative (session union).

dirty_tracking_pause

```
echo "<PID>" > /proc/driver/nvidia-vm/dirty_tracking_pause
```

Acquires query barrier (locks ISRs, drains fault buffer); sets active=false. Faults still serviced for forward progress but no dirty inserts emitted.

dirty_tracking_resume

```
echo "<PID>" > /proc/driver/nvidia-vm/dirty_tracking_resume
```

Re-runs downgrade sequence (revoke PTEs, set active=true) without reinitialising tables. Entries accumulated before pause are preserved.

Implementation

Procfs Interface: Query Controls

dirty_tracking_stop

```
echo "<PID>" > /proc/driver/nvidia-vm/dirty_tracking_stop
```

Restores exact pre-tracking permissions from `dirty_downgraded_pages` masks; destroys live, snapshot, and cumulative tables; flushes stats to `/tmp/vm_dirty_ds_stats.txt`.

dirty_tracking_query_cutover

```
echo "<PID>" > /proc/driver/nvidia-vm/dirty_tracking_query_cutover
```

Acquires query barrier; atomically swaps live → snapshot and installs a fresh live table; increments epoch counter; sets `snapshot_pending=true`.

dirty_tracking_query_dump

```
cat /proc/driver/nvidia-vm/dirty_tracking_query_dump
```

Emits frozen snapshot as `0x<addr> <timestamp_ns>` lines. Delta: snapshot only. Cumulative: merge into session-union (first-write-wins) then emit union. Overflow-safe: snapshot retained and replayed on `seq_file` overflow.

Implementation

Procfs Interface: Instrumentation and Backend

dirty_ds_stats_toggle

```
echo "enable|disable" > /proc/driver/nvidia-vm/dirty_ds_stats_toggle
```

enable: resets all atomic counters and starts accumulation. disable: halts accumulation. Covers insert, lookup, for-each, init, destroy, and callback timing.

dirty_ds_stats

```
cat /proc/driver/nvidia-vm/dirty_ds_stats
```

Prints per-operation call count, average latency (ns), and average rwsem wait time. Also reports average latency for invalidate, activate-now, and barrier-end callbacks.

dirty_tracking_hint

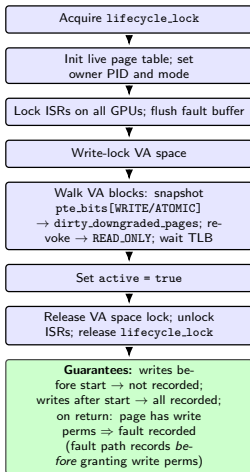
```
echo "WRITE_SEQ|WRITE_RAND" > /proc/driver/nvidia-vm/dirty_tracking_hint
```

Switches backend before a session. WRITE_SEQ → sorted vector (sequential workloads); WRITE_RAND → nested bitmap (random workloads). Rejected if any table is initialised.

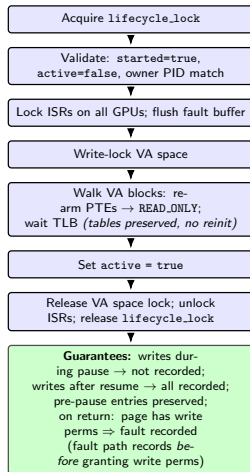
Design Choices

Determinism in start and resume

dirty_tracking_start



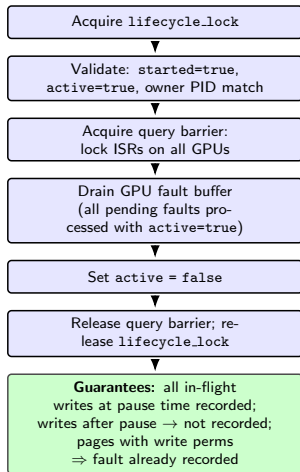
dirty_tracking_resume



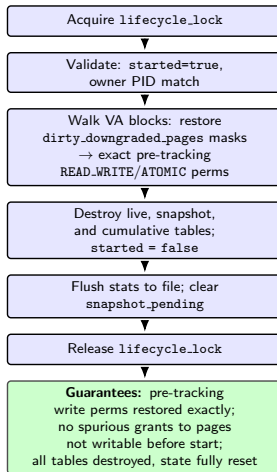
Design Choices

Determinism in pause and stop

dirty_tracking_pause



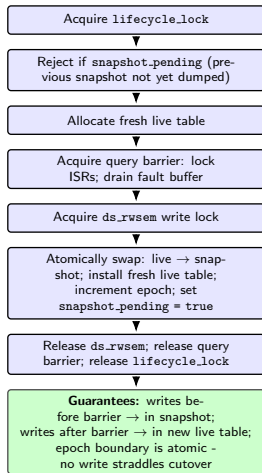
dirty_tracking_stop



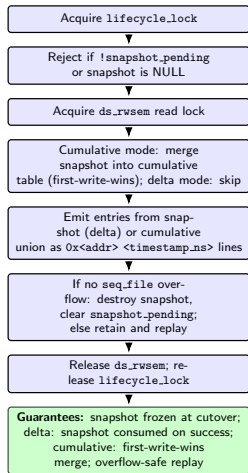
Design Choices

Atomicity in query_cutover and query_dump

dirty_tracking_query_cutover



dirty_tracking_query_dump



Design Choices

Handling Prefetch (`cudaMemPrefetchAsync`)

- `cudaMemPrefetchAsync` migrates pages to the GPU without a fault - it calls into the migration path directly and maps pages `READ_WRITE_ATOMIC`, bypassing `compute_new_permission` and making any subsequent write **invisible** to the tracker
- Fix: in `uvm_migrate.c`, when dirty tracking is started and the destination is a GPU, the migration path forces `prot = UVM_PROT_READ_ONLY` instead of `READ_WRITE_ATOMIC` for all prefetched pages
- The first GPU write to a prefetched page then generates a write fault, which is intercepted by the fault handler - the page and its timestamp are recorded, and write permissions are restored exactly as they would be for a demand-paged write
- This preserves the core invariant: **a page can only acquire write permissions by going through the recorded fault path**, regardless of whether it arrived via demand-paging or prefetch

Design Choices

Query Modes

Design Choices

Concurrency Design

Data Structures and Associated Operations

Overview

Correctness Testing

Summary

Performance Testing

Summary

- **Automatic backend selection:** PTX store instructions reveal sequential vs. random write patterns; this can drive automatic dispatch to the correct backend, replacing the manual `dirty_tracking_hint`
- **Write-reachable VA space narrowing:** NVVM IR alias analysis can identify `cudaMallocManaged` arguments; at `start/resume`, only reachable VA blocks need downgrading, reducing barrier latency at large working-set sizes
- **Write-heat aware eviction:** Epoch-frequency counts from successive dumps can bias UVM's LRU scorer to keep write-hot pages on-device and evict clean pages, reducing re-fault frequency under memory pressure
- **Adaptive cutover for live migration:** Insert counts from `dirty_ds_stats` expose the per-epoch write rate; a migration controller can defer `query_cutover` until the write rate drops below transfer bandwidth, improving pre-copy convergence
- **Dirty ratio as a scheduling signal:** Pages dirtied per epoch divided by allocation size indicates compute phase; checkpoint and migration controllers can adapt inter-round intervals entirely from user space via the existing `procfs` interface

- The GPU fault path is a natural and sufficient instrumentation point. Dirty tracking does not require hardware extensions, compiler changes, or application modification
- Forcing pages to `READ_ONLY` and recording on the first write fault is the right primitive: it gives exact, timestamped, per-page write identity at the cost only of writes, not reads
- Strong lifecycle guarantees matter more than raw performance. The determinism of start, pause, resume, and cutover is what makes the system trustworthy enough to drive live migration and checkpointing
- Overhead is fundamentally proportional to write-fault density; compute-bound kernels that amortise faults over long runtimes see negligible cost, which is exactly the workload profile that benefits most from dirty tracking